

Backpropagation Through Mean Squared Error

A worked example showing the forward pass, chain rule, gradient, and weight update

A clean way to introduce backpropagation is to start with one prediction, one target, and one weight. That makes the chain rule completely visible.

$$x \rightarrow w \rightarrow \hat{y} \rightarrow \text{loss}$$

Let the tiny model be:

$$\hat{y} = wx$$

For one training example, mean squared error reduces to:

$$L = (\hat{y} - y)^2$$

Symbol	Meaning	In this example
y	correct target	10
$\hat{y}$	model prediction	computed from wx
L	loss	squared prediction error

1. Forward Pass

Suppose:

$$x = 3, \quad w = 2, \quad y = 10$$

The model makes its prediction:

$$\hat{y} = wx = (2)(3) = 6$$

The correct answer is 10, so the squared loss is:

$$L = (6 - 10)^2 = 16$$

Forward-pass result

The network predicted 6 when the target was 10, producing a loss of 16.

2. Backward Pass: How Does the Weight Affect the Loss?

Now ask the training question: If we change w a little bit, how will the loss change? That quantity is the derivative of the loss with respect to the weight:

$$\partial L / \partial w$$

But w does not directly determine L . It first changes the prediction $\hat{y}$, and the prediction then changes the loss:

$$w \rightarrow \hat{y} \rightarrow L$$

So the chain rule gives:

$$\partial L / \partial w = (\partial L / \partial \hat{y})(\partial \hat{y} / \partial w)$$

Loss with respect to the prediction

Because

$$L = (\hat{y} - y)^2$$

we have

$$\partial L / \partial \hat{y} = 2(\hat{y} - y)$$

Substitute $\hat{y} = 6$ and $y = 10$:

$$\partial L / \partial \hat{y} = 2(6 - 10) = -8$$

Interpretation: the negative sign says that increasing the prediction would reduce the loss. That makes sense because the current prediction, 6, is too low.

Prediction with respect to the weight

Because

$$\hat{y} = wx$$

the derivative of the prediction with respect to the weight is

$$\partial \hat{y} / \partial w = x = 3$$

Interpretation: near the current value, a small increase in w changes the prediction by about three times as much, because the input x is 3.

3. Apply the Chain Rule

Multiply the two local derivatives:

$$\partial L / \partial w = (\partial L / \partial \hat{y})(\partial \hat{y} / \partial w)$$

$$\partial L / \partial w = (-8)(3) = -24$$

Gradient for the weight: -24

The gradient tells us the direction and sensitivity of the loss with respect to w . Because the gradient is negative, increasing w should reduce the loss.

4. Gradient Descent Updates the Weight

Let the learning rate be:

$$\eta = 0.01$$

Gradient descent updates the weight using:

$$w_{\text{new}} = w - \eta(\partial L / \partial w)$$

Substitute the current values:

$$w_{\text{new}} = 2 - (0.01)(-24) = 2.24$$

Run the model forward again with the new weight:

$$\hat{y}_{\text{new}} = (2.24)(3) = 6.72$$

The new loss is:

$$L_{\text{new}} = (6.72 - 10)^2 = 10.76$$

The loss decreased

The training step moved the loss from 16 to 10.76. The model changed its weight in a direction that made the prediction better.

5. What Actually Backpropagated?

During the forward pass, values move from the parameters toward the loss. During the backward pass, derivatives move in the opposite direction.

Forward pass	Backward pass
$w \rightarrow \hat{y} \rightarrow L$	$L \rightarrow \hat{y} \rightarrow w$
Compute prediction and loss	Compute derivatives / gradients

Mathematically, the key step is:

$$\partial L / \partial w = (\partial L / \partial \hat{y})(\partial \hat{y} / \partial w)$$

Backpropagation repeatedly applies this same chain-rule idea through a larger network to determine how much each weight contributed to the final error.

6. Mean Squared Error for Several Training Examples

For n predictions, mean squared error is:

$$L = (1/n) \sum_i (\hat{y}_i - y_i)^2$$

The derivative with respect to one prediction is:

$$\partial L / \partial \hat{y}_i = (2/n)(\hat{y}_i - y_i)$$

So MSE begins the backward pass by quantifying, for each output, how far the prediction was from the target and in which direction the prediction needs to move.

The Big Picture

Prediction $\rightarrow$ MSE $\rightarrow$ output gradient $\rightarrow$ chain rule $\rightarrow$ gradients for every weight

Core idea

The loss tells us how wrong the network was. Backpropagation tells us how each weight affected that error. Gradient descent then uses those gradients to change the weights.